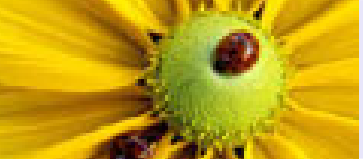
Linguaggi di Programmazione I – Lezione 3

Prof. Marco Faella
mailto://m.faella@unina.it
http://wpage.unina.it/mfaella
Materiale didattico elaborato con i Proff. Sette e Bonatti

18 marzo 2024

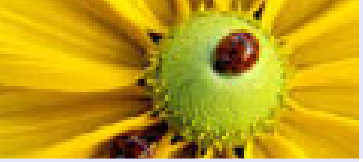


Panoramica della lezione

Procedure come astrazioni

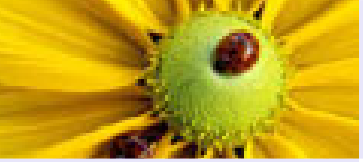
Propagazione dei data object

Bibliografia



Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

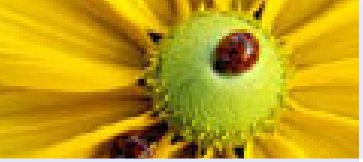
Procedure come astrazioni



Procedure

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Procedure sono astrazioni di parti di programma in unità di esecuzione più piccole (invocazioni), in modo da nascondere i dettagli irrilevanti ai fini del loro (ri-)uso.



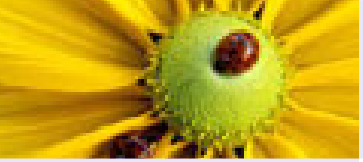
Procedure

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Procedure sono astrazioni di parti di programma in unità di esecuzione più piccole (invocazioni), in modo da nascondere i dettagli irrilevanti ai fini del loro (ri-)uso.

Vantaggi:

- Programmi più semplici da scrivere, leggere o modificare; suddivisione dei compiti in ogni brano di programma; progettazione top-down.



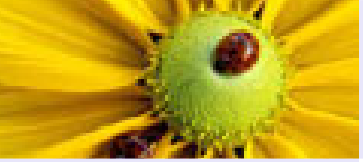
Procedure

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Procedure sono astrazioni di parti di programma in unità di esecuzione più piccole (invocazioni), in modo da nascondere i dettagli irrilevanti ai fini del loro (ri-)uso.

Vantaggi:

- Programmi più semplici da scrivere, leggere o modificare; suddivisione dei compiti in ogni brano di programma; progettazione top-down.
- Unità di programmi indipendenti o con dipendenze ben specificate a livello più alto.



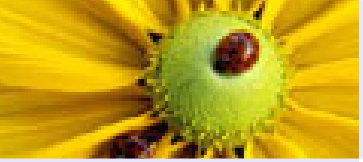
Procedure

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Procedure sono astrazioni di parti di programma in unità di esecuzione più piccole (invocazioni), in modo da nascondere i dettagli irrilevanti ai fini del loro (ri-)uso.

Vantaggi:

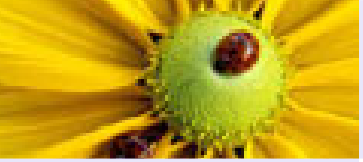
- Programmi più semplici da scrivere, leggere o modificare; suddivisione dei compiti in ogni brano di programma; progettazione top-down.
- Unità di programmi indipendenti o con dipendenze ben specificate a livello più alto.
- Riusabilità di brani di programmi; riduzione errori.



Astrazione procedurale

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

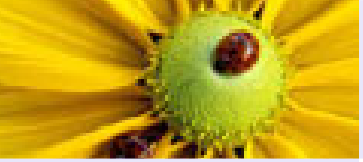
- Se si distinguono come unità di esecuzione, in ordine crescente di complessità, *espressioni*, *enunciati* (statement), *blocchi*, *programmi*, allora si definisce



Astrazione procedurale

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

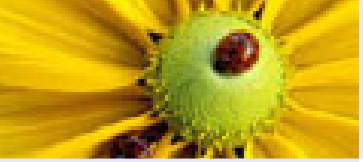
- Se si distinguono come unità di esecuzione, in ordine crescente di complessità, *espressioni*, *enunciati* (statement), *blocchi*, *programmi*, allora si definisce
- **astrazione procedurale** la rappresentazione di una unità di esecuzione attraverso un'altra unità più semplice (l'invocazione)



Astrazione procedurale

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

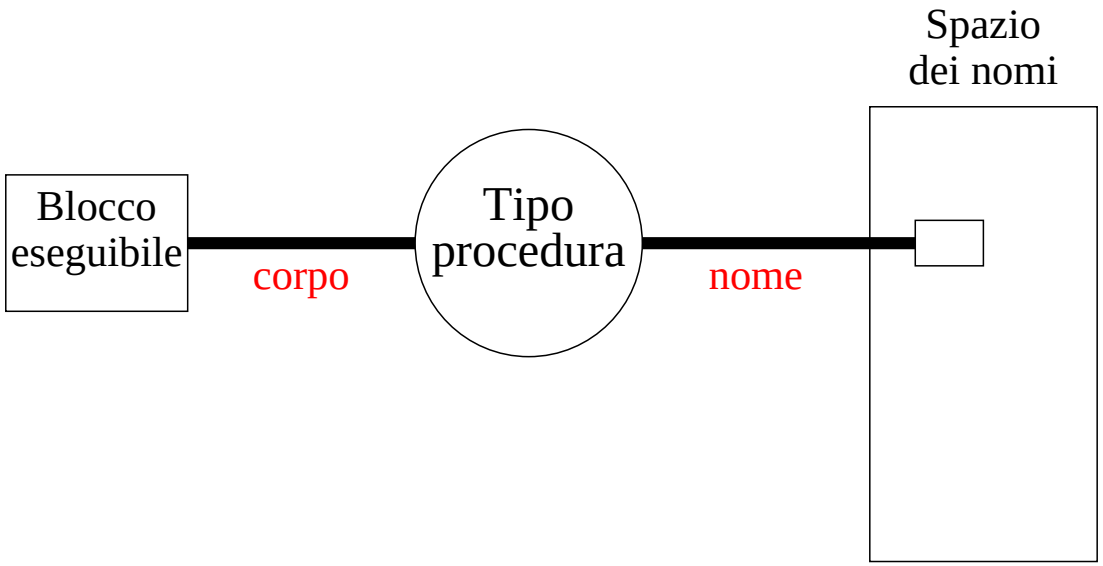
- Se si distinguono come unità di esecuzione, in ordine crescente di complessità, *espressioni*, *enunciati* (statement), *blocchi*, *programmi*, allora si definisce
- **astrazione procedurale** la rappresentazione di una unità di esecuzione attraverso un'altra unità più semplice (l'invocazione)
- In pratica è la rappresentazione di un blocco attraverso un enunciato o una espressione.

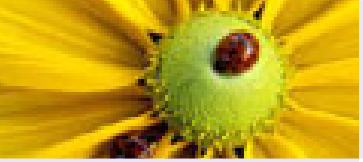


Dichiarazione di procedura

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

- Causa la generazione di un legame tra il nome della procedura e il corpo della procedura (oltre a parametri, tipo di ritorno, etc.)
- Chiamiamo questo legame un oggetto "Tipo procedura"
- Il legame viene stabilito durante la compilazione

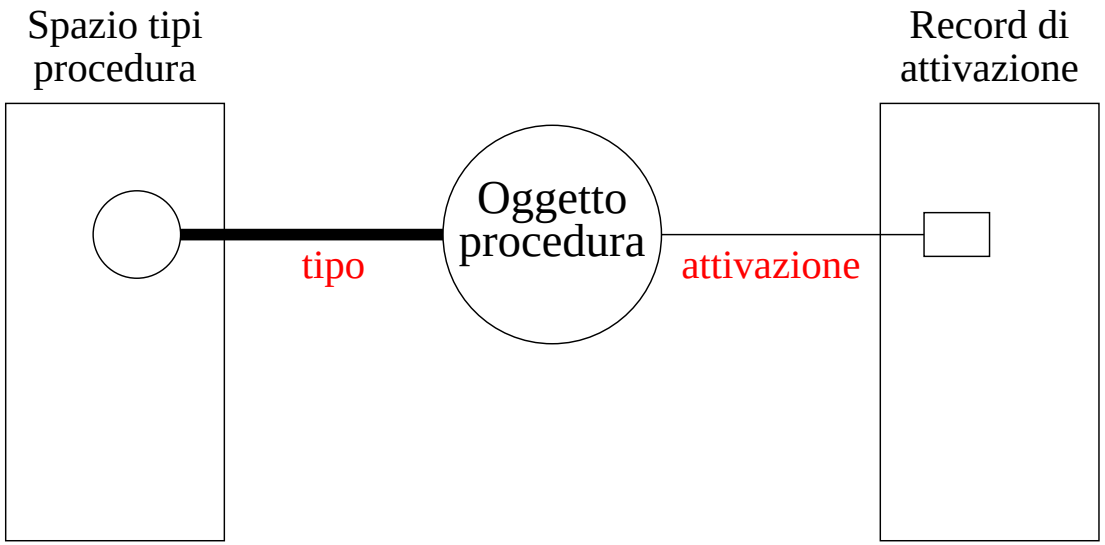


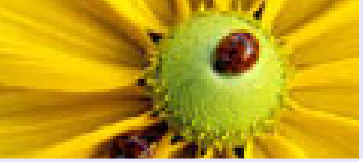


Invocazione di una procedura

Procedure come astrazioni
Procedure Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

- Causa la generazione di un legame tra un Tipo procedura e un Record di attivazione
- Chiamiamo questo legame un "Oggetto procedura"
- Il legame avviene durante l'esecuzione, nel momento in cui c'è l'invocazione della procedura
- Ogni invocazione diversa della stessa procedura causa la generazione di un nuovo "Oggetto procedura" con lo stesso legame di tipo, ma con diverso record di attivazione

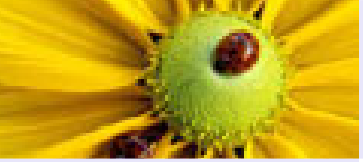




Ambiente di esecuzione

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Analogamente a quanto avveniva per un blocco in-line il **record di attivazione (RdA)** rappresenta l'intero ambiente di esecuzione di una procedura o funzione



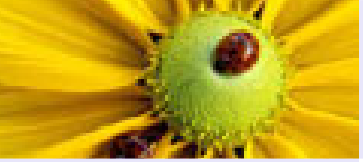
Ambiente di esecuzione

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Analogamente a quanto avveniva per un blocco in-line il **record di attivazione (RdA)** rappresenta l'intero ambiente di esecuzione di una procedura o funzione
A differenza del blocco anonimo, una procedura o funzione...

- Ha un nome e può essere invocata in vari punti del programma
- Può avere parametri
- Può avere un valore di ritorno (se funzione)

Quindi, l'RdA di una funzione contiene più informazioni di quello di un blocco in-line



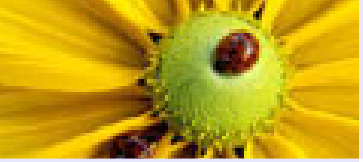
Ambiente di esecuzione

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

L'RdA di una funzione contiene:

1. Puntatore di catena dinamica (link al precedente RdA)
2. Puntatore di catena statica (new!) (opzionale, serve per scoping statico)
3. Indirizzo di ritorno (new!)
4. Indirizzo del risultato (new!)
5. Parametri (new!)
6. Ambiente locale (variabili locali e spazio per risultati intermedi)

L'ambiente non locale di una funzione viene recuperato tramite il puntatore di catena dinamica (scoping dinamico) o statica (scoping statico)

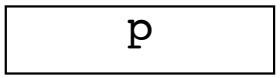


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

p: prima di chiamare s




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

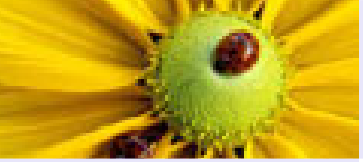
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

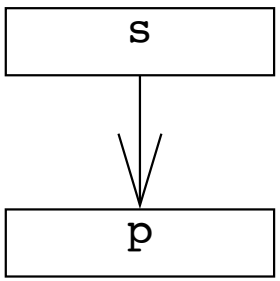


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

s: prima di chiamare r




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

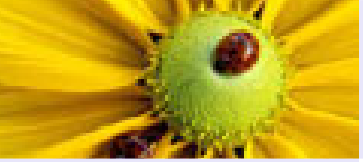
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

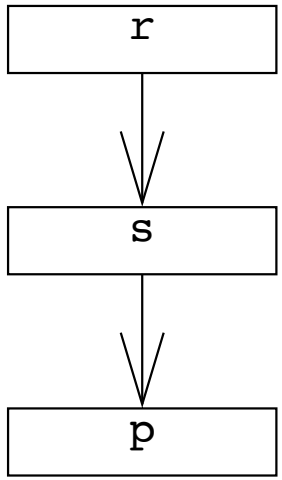


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

r: prima di chiamare r



XS

```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

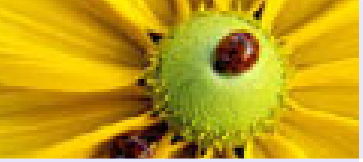
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

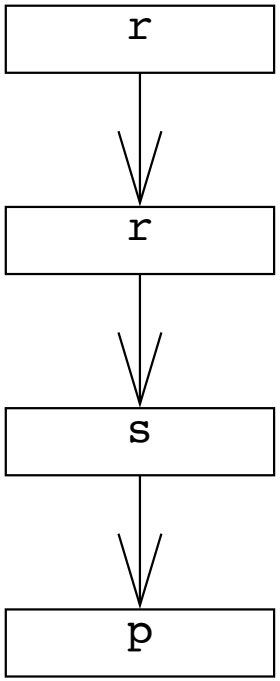


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

r: prima di chiamare q




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

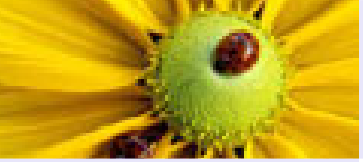
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

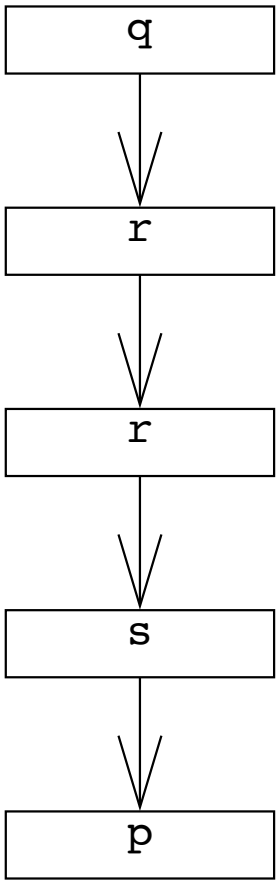


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

q: prima di terminare




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

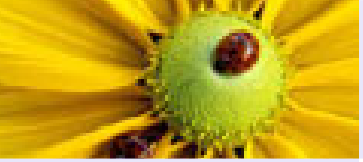
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

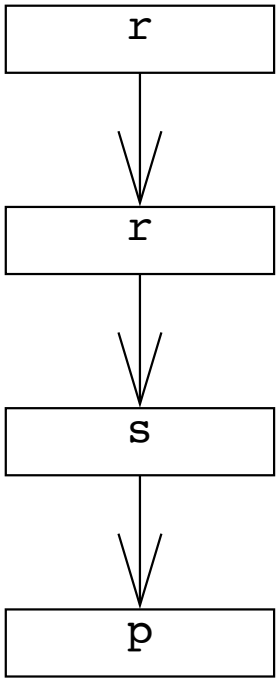


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

r: prima di terminare




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

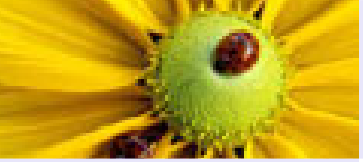
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

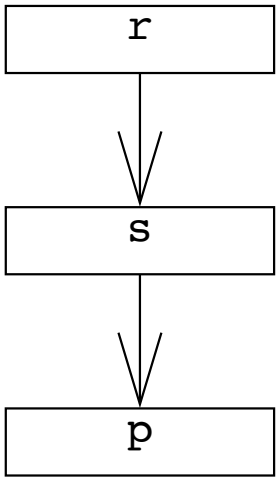


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

r: prima di terminare




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

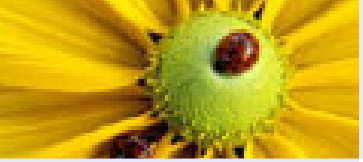
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

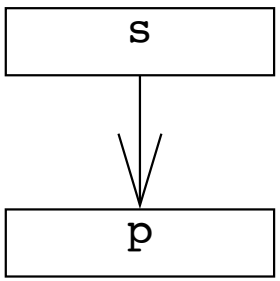


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

s: prima di q




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

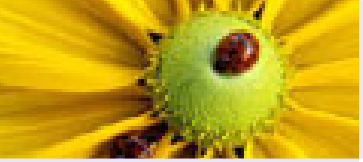
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

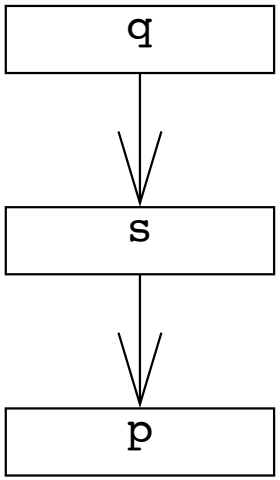


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

q: prima di terminare




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

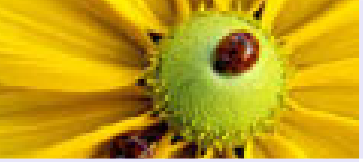
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

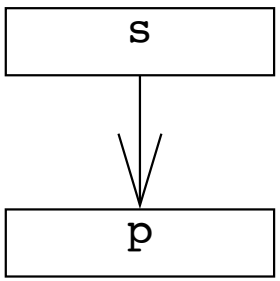


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

s: prima di terminare




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

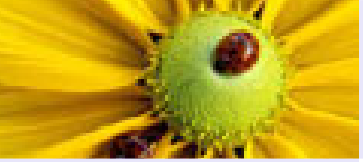
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

  s

end.
```

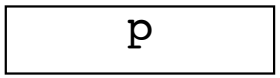


Esempio

Procedure come astrazioni
Procedure
Astrazione procedurale
Dichiarazione
Invocazione di . . .
Ambiente di . . .
Ambiente di . . .
Esempio
Propagazione dei data object
Bibliografia

Stack esecuzione

p: prima di terminare




```
program p;
var i;

  procedure q;
  begin
    ...
  end;

  procedure r;
  begin
    i:= i-1;
    if i>0 then
      r
    else
      q
  end;

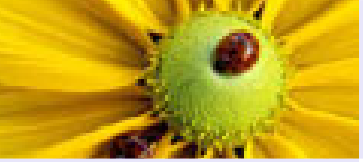
  procedure s;
  begin
    r;
    q
  end;

begin

  i:= 2;

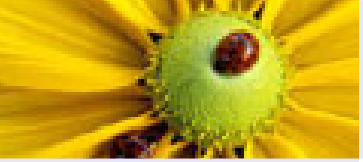
○ s

● end.
```



Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

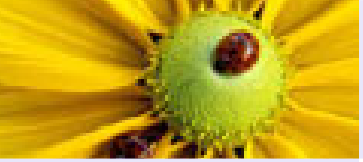
Propagazione dei data object



Ambiente non locale

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

- L'*ambiente non locale* è l'insieme di tutti quei nomi a cui la funzione può riferirsi e che non sono dichiarati in quella funzione
- La provenienza di questi data object propagati dipende dal linguaggio

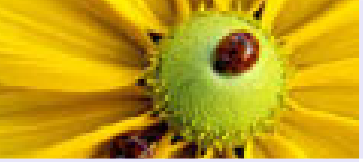


Ambiente non locale

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

- L'*ambiente non locale* è l'insieme di tutti quei nomi a cui la funzione può riferirsi e che non sono dichiarati in quella funzione
- La provenienza di questi data object propagati dipende dal linguaggio
- Tre tipologie di propagazione (scoping):

  1. Propagazione in ambito statico (scoping statico). In questo caso l'ambiente non locale di una procedura è propagato dal programma o dalla procedura che la contiene sintatticamente: propagazione di posizione.

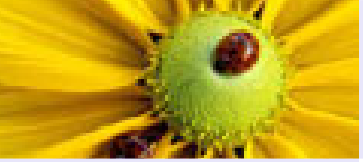


Ambiente non locale

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

- L'*ambiente non locale* è l'insieme di tutti quei nomi a cui la funzione può riferirsi e che non sono dichiarati in quella funzione
- La provenienza di questi data object propagati dipende dal linguaggio
- Tre tipologie di propagazione (scoping):

1. Propagazione in ambito statico (scoping statico). In questo caso l'ambiente non locale di una procedura è propagato dal programma o dalla procedura che la contiene sintatticamente: propagazione di posizione.
2. Propagazione in ambito dinamico (scoping dinamico). In questo caso l'ambiente non locale di una procedura è propagato dal programma o dalla procedura chiamante.

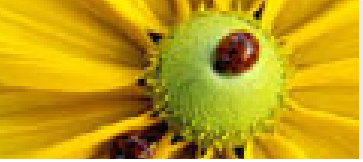


Ambiente non locale

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

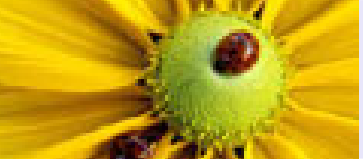
- L'*ambiente non locale* è l'insieme di tutti quei nomi a cui la funzione può riferirsi e che non sono dichiarati in quella funzione
- La provenienza di questi data object propagati dipende dal linguaggio
- Tre tipologie di propagazione (scoping):

1. Propagazione in ambito statico (scoping statico). In questo caso l'ambiente non locale di una procedura è propagato dal programma o dalla procedura che la contiene sintatticamente: propagazione di posizione.
2. Propagazione in ambito dinamico (scoping dinamico). In questo caso l'ambiente non locale di una procedura è propagato dal programma o dalla procedura chiamante.
3. Nessuna propagazione (in generale, l'uso di ambienti non locali è scoraggiato perché produce effetti collaterali non facilmente prevedibili)



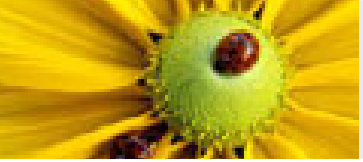
Implementazione

- Lo scoping viene realizzato inserendo nell'RdA un puntatore all'RdA della funzione da cui vengono propagati i data object



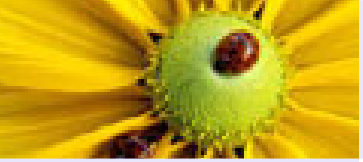
Implementazione

- Lo scoping viene realizzato inserendo nell'RdA un puntatore all'RdA della funzione da cui vengono propagati i data object
  - Scoping statico: viene usato il puntatore di catena statica
  - Scoping dinamico: viene usato il puntatore di catena dinamica
- Se viene richiesto l'accesso ad un dato che non è definito localmente, esso viene ricercato ricorsivamente seguendo la catena appropriata



Implementazione

- Lo scoping viene realizzato inserendo nell'RdA un puntatore all'RdA della funzione da cui vengono propagati i data object
  - Scoping statico: viene usato il puntatore di catena statica
  - Scoping dinamico: viene usato il puntatore di catena dinamica
- Se viene richiesto l'accesso ad un dato che non è definito localmente, esso viene ricercato ricorsivamente seguendo la catena appropriata

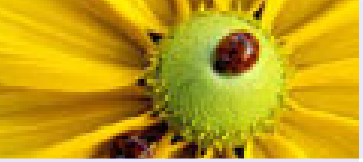


Il caso dei blocchi in-line

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

L'RdA di un blocco in-line non contiene il puntatore di catena statica (anche in presenza di scoping statico) perché viene eseguito sempre nel contesto del blocco in cui è contenuto sintatticamente

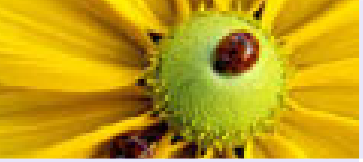
Quindi, l'eventuale puntatore di catena statica *coinciderebbe con quello di catena dinamica*



Il caso del C

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

- Scoping statico
- Non supporta funzioni annidate
- Solo blocchi in-line (anonimi) annidati
- Quindi, non serve il puntatore di catena statica
- Lo scope non locale contiene: i blocchi esterni a quello corrente, poi la funzione corrente e infine lo scope globale



Scoping statico con procedure annidate

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

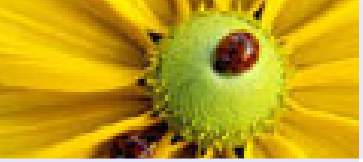
Pseudo-codice:

```
program p;
  var a, b, c: integer;

  procedure q;
    var a, c: integer;
    procedure r;
      var a: integer;
      begin r          {variabili: a da r; b da p; c da q;
        ...             procedure: q ed s da p; r da q}
      end r;
    begin q            {variabili: a da q; b da p; c da q;
       ...              procedure: q ed s da p; r da q}
    end q;

  procedure s;
    var b: integer;
    begin s            {variabili: a da p; b da s; c da p;
      ...              procedure: q ed s da p}
    end s;

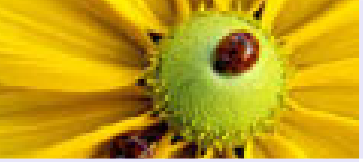
begin p                {variabili: a, b, c da p;
  ...                   procedure: q, s da p}
end p.
```



Scoping statico (2)

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

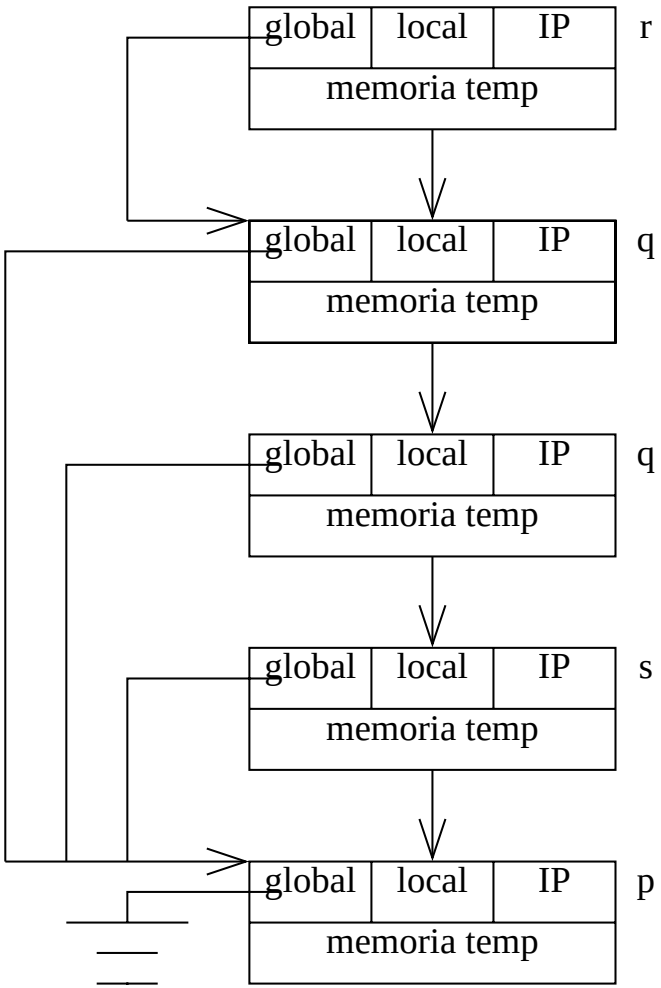
Supponendo una sequenza di attivazione (`p`, `s`, `q`, `q`, `r`), lo stack di esecuzione ha questa forma:

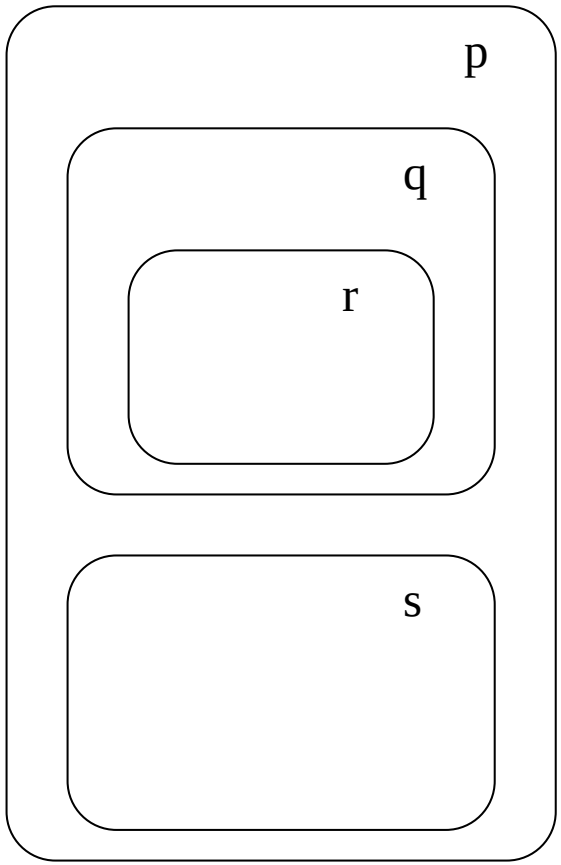


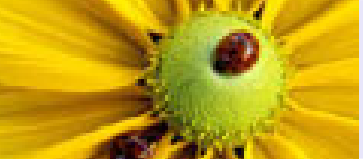
Scoping statico (2)

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

Supponendo una sequenza di attivazione (`p`, `s`, `q`, `q`, `r`), lo stack di esecuzione ha questa forma:



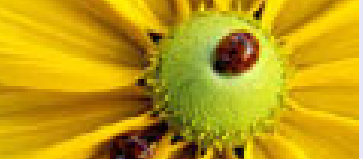




Osservazioni su scoping dinamico

Nello scoping dinamico:

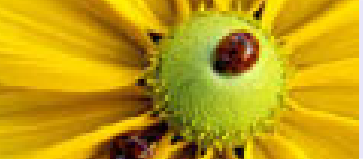
- il puntatore di catena statica non è necessario
- è sufficiente il puntatore di catena dinamica



Osservazioni su scoping dinamico

Nello scoping dinamico:

- il puntatore di catena statica non è necessario
- è sufficiente il puntatore di catena dinamica
- Vantaggio: è possibile trasmettere informazioni *indirettamente* a un'altra procedura (invece di doverle propagare con parametri)

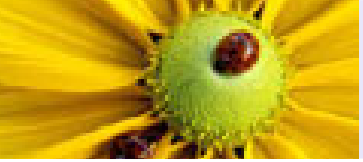


Osservazioni su scoping dinamico

Nello scoping dinamico:

- il puntatore di catena statica non è necessario
- è sufficiente il puntatore di catena dinamica

- Vantaggio: è possibile trasmettere informazioni *indirettamente* a un'altra procedura (invece di doverle propagare con parametri)
- Svantaggio: è praticamente impossibile la determinazione dell'ambiente di esecuzione di una procedura durante la scrittura del codice sorgente.

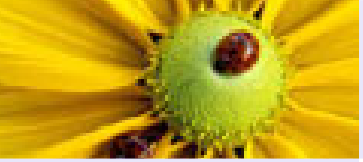


Osservazioni su scoping dinamico

Nello scoping dinamico:

- il puntatore di catena statica non è necessario
- è sufficiente il puntatore di catena dinamica

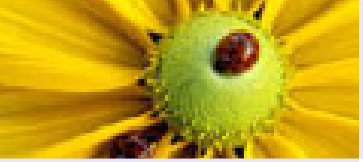
- Vantaggio: è possibile trasmettere informazioni *indirettamente* a un'altra procedura (invece di doverle propagare con parametri)
- Svantaggio: è praticamente impossibile la determinazione dell'ambiente di esecuzione di una procedura durante la scrittura del codice sorgente.



Esempio di scoping dinamico

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia


```
program p;
  var a: integer;
  procedure q;
    begin q              {vars: a da p o da r; procs: q, r da p}
      ...
    end q;
  procedure r;
    var a: integer;
    begin r              {vars: a da r; procs: q, r da p}
      ...
    end r;
begin p                  {vars: a da p; procs: q, r da p}
  ...
end p.
```



Esempio di scoping dinamico

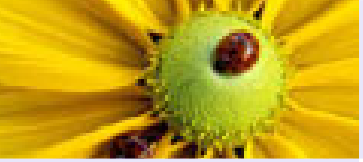
Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

Linguaggio BASH (shell Linux):

```
> x=3
> echo $x
3

> func1 () { echo "in func1: $x"; }
> func1
in func1: 3

> func2 () { local x=9; func1; }
> func2
in func1: 9
```



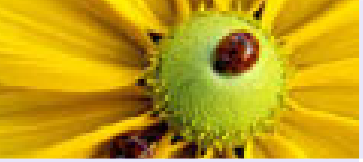
Linguaggi con procedure annidate

Procedure come astrazioni
Propagazione dei data object
Ambiente non locale
Implementazione
Il caso dei blocchi in-line
Il caso del C
Scoping statico con procedure annidate
Scoping statico (2)
Osservazioni su scoping dinamico
Esempio di scoping dinamico
Esempio di scoping dinamico
Linguaggi con procedure annidate
Bibliografia

I seguenti linguaggi supportano l'annidamento di procedure/funzioni:

- I linguaggi funzionali (ML, Scheme, Common LISP, etc.)
- Javascript
- Kotlin
- C# (dalla versione 7.0)
- Estensione GCC del C

Altri linguaggi OO, come Java e C++, supportano l'annidamento di *classi*



Bibliografia

Procedure come astrazioni
Propagazione dei data object
Bibliografia
Bibliografia

Linguaggi di Programmazione, principi e paradigmi, di Gabbrielli e Martini (2a edizione):

- Capitolo 7 ("La gestione della memoria")
- Capitolo 9 ("Astrarre sul controllo")

Type equivalence
+
evoluzione dei sistemi di tipi
+
classificazione dei linguaggi di programmazione

Ritorno al passato

All'inizio esistevano solo i tipi elementari

- E nessuna gerarchia di classi...

Poi sono stati introdotti i tipi user-defined

- Ancora niente classi
- Inizialmente semplici ridenominazioni di tipi elementari oppure nomi di record

Un assegnamento x = y (o x := y) o un passaggio di parametri quando era valido?

Lo stabilisce la nozione di *type equivalence* adottata dal linguaggio

Forme di equivalenza

Name equivalence

I tipi di x e y devono avere lo stesso nome

cioè essere lo stesso tipo

Structural equivalence

I tipi di x e y devono avere la stessa rappresentazione interna

Apparentemente più snello e flessibile, in realtà aumenta la possibilità di errori

```
Euro x;
Dollar y;
z = x+y ;      // che senso ha?
```

Esempi

Pascal: name equivalence

C (e C++): entrambe!

Quasi sempre *structural* tranne che per le *struct*

```
typedef int money;
typedef int apples;

typedef struct{ int val; } S1;
typedef struct{ int val; } S2;

int main() {
	money x=0;
	apples y=0;
	int z = x+y; // non fa una piega
	S1 a;
	S2 b;
	a = b;   // errore di compilazione!
```

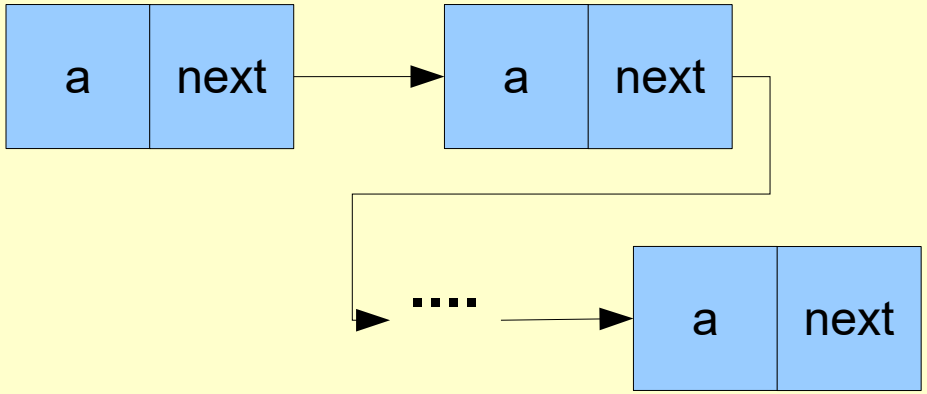
Ecco perché

Verificare se due struct sono strutturalmente equivalenti richiede di verificare una proprietà chiamata *bisimulazione*

Caso semplice: sia S1 che S2 generano tutte le catene come quella a destra

```
struct S1 { int a; struct S1* next; };

struct S2 { int a; struct S3* next; };
struct S3 { int a; struct S2* next; };
```



Complicazioni in presenza di record varianti (union/datatype)

Compatibilità di tipi

Con l'avvento dei linguaggi a oggetti l'equivalenza viene rimpiazzata da *compatibilità*/*assegnabilità*

Qualunque sottotipo di T è assegnabile a T

L’equivalenza è simmetrica

L’assegnabilità (in Java) è anti-simmetrica

Compatibilità di tipi

Nei linguaggi O.O. più comuni la compatibilità è basata su nome piuttosto che struttura

Due classi con nomi diversi sono diverse

- Anche se hanno gli stessi attributi e gli stessi metodi

Inoltre una classe per essere sottotipo di un'altra deve essere esplicitamente dichiarata tale

- In Java, keyword extends e implements

La struttura non conta neanche in questo caso

Un linguaggio OO con equivalenza strutturale è OCaml

Compatibilità di tipi

Linguaggio	**Name equivalence**	**Structural equivalence**
C	Sì (struct)	Sì (typedef)
Java	Sì (name *compatibility, assegnabilità*)	No
ML	Sì (datatype)	Sì (type)

Evoluzione dei sistemi di tipi

- Tipi di dato elementari
- Tipi user-defined (ad es. Pascal, C)
  - Solo strutture dati
- Interfacce e tipi di dato astratti (ad es. Modula, Ada)
  - Encapsulation: strutture dati accessibili solo attraverso specifiche procedure
  - Modificatori di accesso
  - Disaccoppiamento interfaccia/implementazione
- Gerarchie di tipi
  - Ed ecco finalmente i linguaggi O.O.

Caratteristiche utili alla classificazione dei linguaggi

- Paradigma di riferimento
  - imperativo, OO, funzionale, logico
- Scoping (statico/dinamico)
- Gestione della memoria
  - Allocazione dinamica, presenza garbage collection, ...
- Sistema di tipi
  - Forte/debole, statico/dinamico, encapsulation, equivalence/compatibility, polimorfismo (di 4 tipi), *type inference*...
- Supporto alle eccezioni
  - Eventualmente integrato con type checking (vedi ML)
- Supporto al parallelismo
  - Gestione thread, scambio di messaggi (*Remote Method Invocation*), memory model, procedure asincrone, ...

Caratteristiche utili alla classificazione dei linguaggi

- Naturalmente determinano come si usa al meglio un dato linguaggio
- E quali sono gli errori da evitare
  - Ad es. sapendo cosa il compilatore può o non può fare per voi